

Systems Thinking Project

Acrobot

Phase 5: Reinforcement-Learning Swing-Up

Author: Bhargav
ID: 2025102061

Author: Sreekar
ID: 2025102074

Author: Kaushik
ID: 2025102071

Author: Divijat
ID: 2025102040

November 11, 2026

Contents

1 Introduction **2**

1.1 What is an Acrobot? . . . 2
1.2 Model . . . 2
1.3 Equations of Motion . . . 2
1.4 Equilibria and Linear Model (from Phase 1) . . . 3
1.5 Objective . . . 4

2 Reinforcement-Learning Swing-Up **4**

2.1 Why Reinforcement Learning? . . . 4
2.2 Hybrid Structure . . . 4
2.3 Environment . . . 4
2.4 Reward Design . . . 5
2.5 Algorithm: Soft Actor-Critic (SAC) . . . 5
2.6 Training . . . 6
2.7 Results: Correct Model . . . 6
2.8 Results: Wrong Model . . . 7
2.9 Making the Policy Robust . . . 7
2.10 Discussion . . . 9

3 Simulations and Code **9**

3.1 Training and Evaluation Pipeline . . . 9
3.2 Python Codes . . . 10
3.3 Animation . . . 11

4 Tabulation **12**

5 Conclusion **12**

6 Contributions **12**

1 Introduction

1.1 What is an Acrobot?

An acrobot is a two-link pendulum that hangs from a fixed pivot. The first link (the "shoulder" link) is free to rotate about the pivot; the second link is joined to the end of the first at the "elbow". The only motor sits at the elbow, so the shoulder is *passive*. It is the robot version of a gymnast on a high bar who can only swing the legs at the hips. Because there are two degrees of freedom but only one actuator, the acrobot is a standard example of an *underactuated* system [1], [2].

1.2 Model

The configuration is described by two generalised coordinates: q_1, the angle of link 1 from the downward vertical, and q_2, the angle of link 2 relative to link 1. The full state contains the angles and their rates:

$$\mathbf{q} = \begin{bmatrix} q_1 \\ q_2 \end{bmatrix}, \qquad \mathbf{x} = \begin{bmatrix} q_1 \\ q_2 \\ \dot{q}_1 \\ \dot{q}_2 \end{bmatrix}.$$

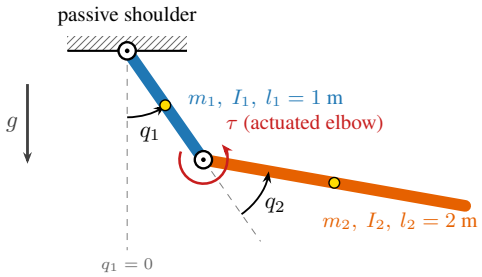


Figure 1: The acrobot: passive shoulder at the pivot, motor torque τ at the elbow; dots mark the centres of mass.

1.3 Equations of Motion

The dynamics (given in the Phase 1 helper and verified in Phase 1) are

$$M(q)\,\ddot{q} + C(q,\dot{q})\,\dot{q} = \tau_g(q) + B\,\tau, \qquad B = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \qquad |\tau| \le 20\,\mathrm{N\,m}. \tag{1}$$

τ **(Input Torque):** the torque of the elbow motor, the only quantity we command. $B = [0,\ 1]^T$ says it acts on the elbow equation only.

$M(q)$ **(Inertia Matrix):** the resistance to angular acceleration. It depends on the elbow angle because the mass distribution changes as the arm folds:

$$M(q) = \begin{bmatrix} 2.663 + 2\cos q_2 & 1.33 + \cos q_2 \\ 1.33 + \cos q_2 & 1.33 \end{bmatrix}, \qquad \det M = 1.7729 - \cos^2 q_2 \ge 0.7729 > 0. \tag{2}$$

$C(q,\dot{q})$ **(Coriolis Matrix):** the coupling forces between the links (centrifugal and Coriolis effects):

$$C(q,\dot{q}) = \sin q_2 \begin{bmatrix} -2\dot{q}_2 & -\dot{q}_2 \\ \dot{q}_1 & 0 \end{bmatrix}. \tag{3}$$

$\tau_g(q)$ **(Gravity Torque):** the torque that gravity exerts on each joint, $\tau_g = -\partial U/\partial q$, with potential energy U:

$$U = -14.715\cos q_1 - 9.81\cos(q_1+q_2), \qquad \tau_g(q) = -9.81\begin{bmatrix} 1.5\sin q_1 + \sin(q_1+q_2) \\ \sin(q_1+q_2) \end{bmatrix}. \tag{4}$$

System Parameters:

$$m_1 = m_2 = 1\,\mathrm{kg}, \qquad l_1 = 1\,\mathrm{m}, \qquad l_2 = 2\,\mathrm{m}, \qquad l_{c1} = 0.5\,\mathrm{m}, \qquad l_{c2} = 1\,\mathrm{m},$$
$$I_1 = 0.083\,\mathrm{kg\,m^2}, \qquad I_2 = 0.33\,\mathrm{kg\,m^2}, \qquad g = 9.81\,\mathrm{m\,s^{-2}}.$$

1.4 Equilibria and Linear Model (from Phase 1)

With $\tau^* = 0$ the equilibria satisfy $\tau_g(q^*) = 0$, i.e. $\sin q_1^* = 0$ and $\sin(q_1^* + q_2^*) = 0$. This gives four postures:

Table 1: The four equilibria

	Posture	(q_1^*, q_2^*)	U [J]	Description
E_1	down–down	$(0, 0)$	-24.525	both links hanging
E_2	down–up	$(0, \pi)$	-4.905	link 1 down, link 2 up
E_3	up–up	$(\pi, 0)$	$+24.525$	fully upright (target)
E_4	up–down	(π, π)	$+4.905$	link 1 up, link 2 down

Linearizing about each equilibrium with $\tilde{x} = x - x^*$ gives

$$\dot{\tilde{x}} = A\tilde{x} + B_\ell\tilde{\tau}, \qquad A = \begin{bmatrix} 0 & I \\ P & 0 \end{bmatrix}, \qquad B_\ell = \begin{bmatrix} 0 \\ r \end{bmatrix}, \tag{5}$$

with $P = M(q^*)^{-1}K(q^*)$ and $r = M(q^*)^{-1}B$, where $K = \partial\tau_g/\partial q$. The transfer functions from τ to the angles are

$$G_1(s) = \frac{r_1 s^2 + (P_{12}r_2 - P_{22}r_1)}{\Delta(s)}, \qquad G_2(s) = \frac{r_2 s^2 + (P_{21}r_1 - P_{11}r_2)}{\Delta(s)}, \tag{6}$$
$$\Delta(s) = s^4 - \mathrm{tr}(P)\,s^2 + \det P.$$

Table 2: Transfer functions at the four equilibria (Phase 1)

	$\Delta(s)$	$N_1(s)$ $(\tau \to q_1)$	$N_2(s)$ $(\tau \to q_2)$
E_1	$s^4 + 42.2410\,s^2 + 186.7719$	$-3.0147\,(s^2 + 4.2103)$	$6.0332\,(s^2 + 5.2595)$
E_2	$s^4 + 8.4025\,s^2 - 186.7719$	$-0.4270\,(s^2 - 29.7273)$	$0.8578\,(s^2 + 7.3982)$
E_3	$s^4 - 42.2410\,s^2 + 186.7719$	$-3.0147\,(s^2 - 4.2103)$	$6.0332\,(s^2 - 5.2595)$
E_4	$s^4 - 8.4025\,s^2 - 186.7719$	$-0.4270\,(s^2 + 29.7273)$	$0.8578\,(s^2 - 7.3982)$

1.5 Objective

Replace the hand-designed Phase 4 swing-up with one learned by reinforcement learning, keep the same Phase 3 LQR for the final catch, and compare the two under $|\tau| \leq 20\,\mathrm{N\,m}$: reliability, speed, control effort, and robustness when the real robot differs from the model.

Phase 4 provides the baseline: an energy-pumping swing-up with partial feedback linearization, gains $k = (20, 2, 3)$, which switches to the Phase 3 LQR ($Q = I$, $R = 0.03$) when $\tilde{x}^T S \tilde{x} < 5$.

2 Reinforcement-Learning Swing-Up

2.1 Why Reinforcement Learning?

The Phase 4 swing-up works, but it is built entirely from the model: the partial feedback linearization needs $M(q)$, $C(q, \dot{q})$ and $\tau_g(q)$, and the energy target needs the exact masses and lengths. Reinforcement learning (RL) takes a different route. A neural-network policy learns *by trial and error in a simulator* which torque to apply, guided only by a score (the reward). No equation of motion is written into the controller.

- **Question:** can a learned swing-up match or beat the hand-designed one under the same $20\,\mathrm{N\,m}$ limit?
- **Fair comparison:** both controllers hand over to the *same* Phase 3 LQR inside the *same* catch zone, so the only difference is who does the swing-up.
- **Extra test:** what happens when the real robot differs from the model both controllers were designed or trained on?

2.2 Hybrid Structure

$$\tau = \begin{cases} \mathrm{sat}\Big(-K\tilde{x}\Big), & \tilde{x}^T S \tilde{x} < c_{sw} = 5 \quad \text{(catch: Phase 3 LQR, } R = 0.03\text{)} \\ 20\,\pi_\theta(o), \quad \pi_\theta(o) \in [-1, 1], & \text{otherwise (learned swing-up, held for 20 ms)} \end{cases} \tag{7}$$

RL only has to deliver the arm into the catch zone; it never has to learn to balance. The part we can prove (the LQR near the top) stays model-based, and the hard nonlinear part is learned. If the arm leaves the zone, control returns to the policy, the same retry logic as Phase 4.

2.3 Environment

The environment follows the Gymnasium interface and uses the given equations of motion (1), integrated with RK4.

Table 3: RL environment

Element	Choice	Reason
Observation o	$[\cos q_1,\ \sin q_1,\ \cos q_2,\ \sin q_2,\ \dot{q}_1/10,\ \dot{q}_2/10]$	no jump at $\pm 180°$; inputs of order 1
Action a	$a \in [-1, 1]$, $\tau = 20\,a$	the torque limit is built in: the policy cannot exceed it
Control step	20 ms (50 Hz), four RK4 steps of 5 ms	realistic control rate
Start	$\lvert\tilde{q}_i(0)\rvert \le 0.6$ rad, $\lvert\dot{q}_i(0)\rvert \le 0.5$ rad/s around hanging	same spread as the Phase 4 robustness test
Episode	15 s; ends early on entering the catch zone	short episodes, one clear goal

2.4 Reward Design

$$r = -\left(\frac{E - E_{top}}{E_{top}}\right)^2 - 0.3\,(1 + \cos q_1) - 0.1\,(1 - \cos q_2) - 0.001\left(\frac{\tau}{20}\right)^2 + 100 \cdot \mathbb{1}[\text{enter catch zone}]. \quad (8)$$

- **Energy term:** the same target as Phase 4. It tells the policy *how much* energy to pump. Without it, RL tends to learn to spin the arm round and round, because a spinning arm passes near the top often.
- **Angle terms:** $1 + \cos q_1$ is 0 when link 1 points up; $1 - \cos q_2$ is 0 when the elbow is straight. They tell the policy *where* to go.
- **Torque term:** a small penalty that discourages wasted effort and bang–bang control.
- **Catch bonus:** +100 for reaching the region where the LQR takes over; the episode then ends.

2.5 Algorithm: Soft Actor-Critic (SAC)

SAC [8] learns a stochastic policy π_θ (the actor) and two action-value networks Q_ϕ (the critics) that maximize reward plus entropy:

$$J(\pi) = \mathbb{E}\Big[\sum_t \gamma^t \big(r_t + \alpha\,\mathcal{H}(\pi(\cdot \mid o_t))\big)\Big]. \quad (9)$$

The entropy term $\mathcal{H}$ keeps the policy exploring, and α is tuned automatically. SAC suits this problem because the torque is continuous and SAC reuses past experience (off-policy), so it needs relatively few simulated swings.

Table 4: Training settings (stable-baselines3 [9])

Setting	Value
Networks	2 hidden layers $\times$ 256 units (actor and critics)
Discount γ, learning rate, batch	0.99, 3×10^{-4}, 256
Replay buffer	300 000 transitions
Parallel environments, seed	4, 0
Training length	10^6 steps = 9 206 episodes, 12.4 min on a laptop CPU

2.6 Training

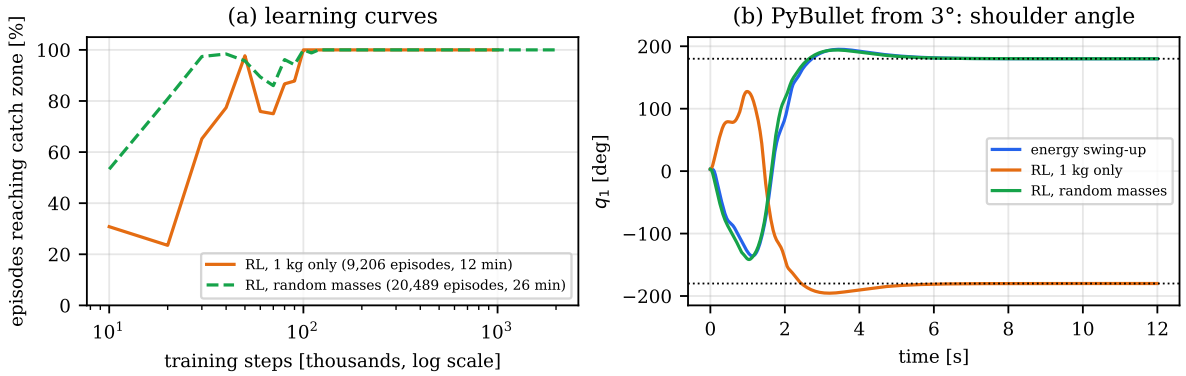


Figure 2: (a) Share of training episodes that reached the catch zone, for the original policy and for the robust policy of Section 2.9 (log scale). (b) PyBullet runs from 3°: the original RL policy swings the other way and ends at $q_1 = -180°$, the same upright pose as $+180°$; the robust policy swings the same way as the energy law.

Observations:

- The policy first reached the catch zone in every episode after about 10^5 steps (72 s of training) and stayed at 100% for the remaining 9×10^5 steps.
- Episodes kept getting shorter (more episodes per 10^4 steps: 54 at 10^5, 110 at 10^6): the policy kept learning to reach the top faster.
- The robust policy (Section 2.9), which faces a different robot in every episode, also first reached 100% after about 10^5 steps. Over its remaining 1.9×10^6 steps it never dropped below 98.8%.

2.7 Results: Correct Model

Both controllers were tested from the same 40 random starts as Phase 4. A start counts as held if it is caught and balanced within 30 s. (This is why the energy controller scores 35 rather than the Phase 4 value of 39, which allowed 40 s.)

Table 5: RL vs. energy swing-up, correct masses ($|\tau| \leq 20\,\mathrm{N\,m}$)

Metric	Energy swing-up	RL swing-up (SAC)
Held (of 40)	35	**40**
Median time to first catch	3.98 s	**1.71 s**
Time at the torque limit	8.8%	**0.1%**
Median elbow work	244.9 J	**144.5 J**
PyBullet from 3°: first catch	2.17 s	**1.76 s**

Observations:

- **Reliability:** RL caught all 40 starts; the energy law caught 35 within 30 s.

- **Speed:** RL reaches the top more than twice as fast (median 1.71 s vs. 3.98 s).
- **Control effort:** RL almost never saturates the motor (0.1% vs. 8.8%) and uses 41% less work. In Phase 4 saturation was what broke the energy law; the policy learned a timing that stays within 20 N m by itself.

2.8 Results: Wrong Model

The real robot never matches the model exactly. The same 40 starts were rerun with both link masses changed by ±20% (inertias scaled with them). Both controllers still assume 1 kg links.

Table 6: Robustness to the link masses (held of 40, within 30 s)

Link masses	Energy swing-up	RL swing-up (SAC)
−20% (0.8 kg)	37	38
correct (1.0 kg)	35	40
+20% (1.2 kg)	**35**	**15**

- **The energy law degrades gracefully** (35–37 of 40 in every case). It is built from physics: it pumps until $E = E_{top}$, so a heavier arm just takes a little longer (median 5.4 s instead of 4.0 s).
- **RL collapses on heavier links** (15 of 40). It practised only on 1 kg links, so it learned the *timing of that particular robot*, not the physics. It still reaches the catch zone in all 40 runs, but in a state from which the LQR cannot hold the heavier arm; it then repeats nearly the same approach (a median of 13 hand-overs in 30 s instead of 2).
- **The fix** is to let the policy practise on many different robots (Section 2.9).

2.9 Making the Policy Robust

A second policy was trained with exactly **two changes**. Everything else (network, reward, catch zone, LQR and SAC settings) is the same, so any difference in the results comes from these two changes.

Table 7: The two changes for the robust policy

	Original policy	**Robust policy**	**Why**
1. Link masses in training	always 1 kg	new random $m_1, m_2 \in [0.7, 1.3]$ kg in every episode	it cannot memorise one robot's timing
2. Observation	the 6 numbers of Table 3	the same 6, plus the missing energy $(E_{top} - E)/E_{top}$	the quantity the energy law steers by
Training length	10^6 steps, 9 206 episodes, 12.4 min	2×10^6 steps, 20 489 episodes, 25.9 min	many robots make a harder task

The extra input is computed from the other six with the nominal 1 kg model, exactly as the energy controller computes it. It therefore adds no new information: it is a hint that makes the important quantity explicit. Change 1 is the real fix (this is called *domain randomization*). By the end of training the robust policy reached the catch zone in 100% of episodes, as the original did.

All three swing-ups were then tested from the same 40 starts with five link masses. Two of them (0.6 and 1.4 kg) lie *outside* the range the robust policy practised on. Every controller still assumes 1 kg links.

Table 8: Held of 40 random starts (within 30 s) for five link masses

True link masses	Energy swing-up	RL, 1 kg only	RL, random masses
0.6 kg (−40%, never practised)	39	2	**40**
0.8 kg (−20%)	37	38	**40**
1.0 kg (correct)	35	**40**	**40**
1.2 kg (+20%)	35	15	**40**
1.4 kg (+40%, never practised)	**33**	3	2

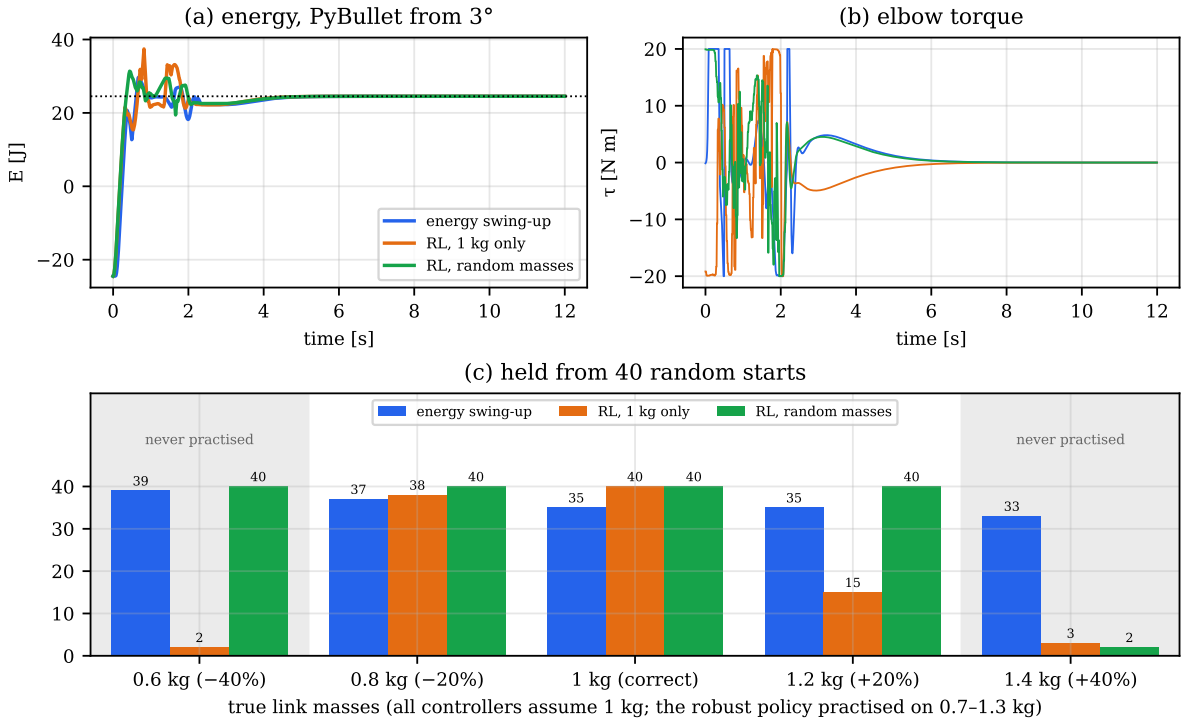


Figure 3: (a) Energy and (b) elbow torque in PyBullet from 3° for the three swing-ups; (c) starts held for five link masses. Shaded: masses the robust policy never practised on.

Observations:

- **Inside the practised range the problem is solved:** 40 of 40 at every mass from 0.8 to 1.2 kg. At +20% the score rises from 15 to 40.
- **No price on the correct model:** the robust policy also holds 40 of 40, with the same median catch time (1.71 s), no saturation (0.0%) and 10% less elbow work (130.0 J vs. 144.5 J). In PyBullet from 3° it catches at 1.89 s (original 1.76 s).

- **Lighter than ever practised (0.6 kg):** still 40 of 40. The original policy managed only 2, and in 25 of its 40 runs it never even reached the catch zone.
- **Much heavier than ever practised (1.4 kg):** only 2 of 40, while the energy law still holds 33. The policy reaches the catch zone in all 40 runs, but the hand-over fails again and again (a median of 12 hand-overs in 30 s, against 1 for the energy law). The energy law also loses many first catches, but each new approach is different, so a later catch usually works; the policy repeats nearly the same approach.
- **Why the hand-over is the weak point:** the reward ends the episode as soon as the arm *enters* the catch zone. The policy is never told whether the LQR then holds the arm, so it never learned to deliver the arm in an easy-to-catch state.

Lesson: a learned controller is reliable over the variety of robots it practised on (here, also somewhat beyond it on the light side). A physics-based law knows *why* it works, so it extrapolates further. Two natural next steps would be to reward holding rather than only arriving (end each episode only after the LQR has held the arm for, say, 1 s) and to widen the mass range.

2.10 Discussion

1. **Why RL is faster and gentler.** The energy law fixes the elbow motion through v and pays for it with saturation. The policy is free to choose any torque profile and is rewarded for reaching the top quickly with little effort, so it found a timing that pumps energy faster without hitting the limit.
2. **Why RL was fragile, and what fixed it.** Its knowledge is implicit in 256×256 hidden units fitted to the robots it practised on. Practising on one robot gave that robot's timing; practising on many made it robust over that range. Nothing in it says "pump until $E = E_{top}$", however, so far outside that range (1.4 kg) it can still fail where the energy law does not.
3. **Guarantees.** The LQR catch has a Lyapunov certificate (Phase 3). The learned part has none: its performance is known only from tests, which is why it was tested on 40 starts, in a second simulator (PyBullet) and with five different link masses.
4. **Hybrid design.** Learning the hard nonlinear part and keeping the provable part near the top gave the best of both: the speed of RL, and a guaranteed final balance.

3 Simulations and Code

3.1 Training and Evaluation Pipeline

The work is in `phase5_rl/`; `bash run_pipeline.sh 1000000` runs every step in order (about 50 minutes on a laptop CPU):

1. **Model check:** the mass-parametric model reproduces the given equations of motion to 10^{-13}.

2. **Training** (`train.py`): SAC for 10^6 steps on 1 kg links, then the robust policy (option `--robust`) for 2×10^6 steps. Both policies are saved in the folder `policies/`, each with its learning curve.

3. **Evaluation** (`evaluate.py`): all three swing-ups from 40 random starts with five link masses (0.6–1.4 kg), plus PyBullet runs; writes `results.json` and the figures above.

4. **Video** (`manim-sim/rl_vs_energy.py`): the explanatory comparison animation.

Libraries: Gymnasium 1.4 (environment interface), stable-baselines3 2.9 (SAC), PyTorch 2.15, PyBullet.

3.2 Python Codes

The files are listed below; the algorithms they implement follow as pseudo-code.

Table 9: Python files (`phase5_rl/`)

File	Purpose
`acrobot_env.py`	Gymnasium environment, mass-parametric model, reward, RL → LQR controller
`train.py`	SAC training; saves the policy and the learning curve
`evaluate.py`	40-start and wrong-mass tests, PyBullet runs, figures
`run_pipeline.sh`	runs model check, training, evaluation and video in order

Algorithm 1 Learned swing-up: environment step and SAC training (`acrobot_env.py`, `train.py`)

function RESET
  $x \leftarrow$ random start near hanging; $m_1, m_2 \leftarrow 1$ kg ▷ robust policy: $m_i \sim U[0.7, 1.3]$ kg
end function
function ENVSTEP(x, action $a \in [-1, 1]$)
  $\tau \leftarrow 20a$; integrate the model with masses m_1, m_2 for 20 ms (four RK4 steps of 5 ms)
  $r \leftarrow -((E - E_{top})/E_{top})^2 - 0.3(1 + \cos q_1) - 0.1(1 - \cos q_2) - 0.001(\tau/20)^2$
  if $\tilde{x}^T S \tilde{x} < 5$ **then**
    $r \leftarrow r + 100$; episode ends ▷ reached the LQR catch zone
  end if
  $o \leftarrow [\cos q_1, \sin q_1, \cos q_2, \sin q_2, \dot{q}_1/10, \dot{q}_2/10]$ ▷ robust policy: append $(E_{top} - E)/E_{top}$
  return o, r
end function
initialise actor π_θ, critics Q_{ϕ_1}, Q_{ϕ_2} (2×256 each), empty replay buffer
for 10^6 environment steps (4 environments in parallel) **do**
  $a \sim \pi_\theta(\cdot \mid o)$; $(o', r) \leftarrow$ ENVSTEP(x, a); store (o, a, r, o') in the buffer
  sample a batch of 256; update the critics towards $r + \gamma(\min_i Q_{\phi_i}(o', a') - \alpha \log \pi_\theta(a' \mid o'))$
  update the actor to maximise $\min_i Q_{\phi_i}(o, a) - \alpha \log \pi_\theta(a \mid o)$; tune α
end for
save π_θ

Algorithm 2 Learned swing-up: RL → LQR controller and evaluation (`acrobot_env.py`, `evaluate.py`)

function RLCONTROLLER(x)
  if $\tilde{x}^T S \tilde{x} < 5$ **then**
    return $\mathrm{sat}(-K\tilde{x})$ ▷ same upright LQR catch
  end if
  every 20 ms: $\tau \leftarrow 20\,\pi_\theta(o)$ (deterministic); hold it in between
  return τ
end function
for link masses $\in \{0.6,\ 0.8,\ 1.0,\ 1.2,\ 1.4\}$ kg **do** ▷ controllers always assume 1 kg
  for each of the 40 random starts, for the energy and both RL controllers **do**
    simulate 30 s; record held, first catch time, torque saturation, elbow work and hand-overs
  end for
end for

3.3 Animation

A 2K Manim animation comparing the RL and energy swing-ups is in `phase5_rl/RL_vs_Energy_SwingUp_2K.mp4`.

4 Tabulation

Table 10: Energy swing-up vs. the two RL swing-ups: summary (40 random starts, held within 30 s)

Criterion	Energy swing-up	RL, 1 kg only	RL, random masses
Held, correct masses	35/40	40/40	40/40
Median time to catch	3.98 s	1.71 s	1.71 s
Time at the torque limit	8.8%	0.1%	0.0%
Median elbow work	244.9 J	144.5 J	130.0 J
Held, masses −40% (never practised)	39/40	2/40	40/40
Held, masses −20%	37/40	38/40	40/40
Held, masses +20%	35/40	15/40	40/40
Held, masses +40% (never practised)	33/40	3/40	2/40
Needs the equations of motion	yes	no (only a simulator)	no (only a simulator)
Explainable / provable	yes (energy argument)	no (tested only)	no (tested only)
Design effort	3 gains + search	reward design + 12 min training	same + 26 min training

5 Conclusion

A swing-up policy learned with Soft Actor-Critic, handing over to the same Phase 3 LQR, outperformed the hand-designed energy swing-up on the model it was trained on. It held 40 of 40 random starts instead of 35, reached the top more than twice as fast (1.7 s vs. 4.0 s), almost never saturated the motor, and used 41% less work. It learned this in 12 minutes from a reward alone, with no equations of motion in the controller.

The first policy paid for this with robustness. With links 20% heavier than in training it held only 15 of 40 starts, while the physics-based law still held 35, because the policy had learned one robot's timing rather than the energy principle. A second policy, trained with only two changes (random link masses between 0.7 and 1.3 kg in every episode, and the missing energy as one extra input), held 40 of 40 at every mass from 0.6 to 1.2 kg at no cost on the correct model. At 1.4 kg, far heavier than anything it practised on, it still failed (2 of 40) where the energy law held 33: a learned controller is reliable over the variety it practised on, while a physics-based law extrapolates further. The hybrid design (learn the hard nonlinear swing-up, keep the provable LQR catch) gives the speed of learning with a guaranteed final balance.

6 Contributions

1. Report compilation – _____
2. RL problem and environment design (Sections 2.1–2.4) – _____
3. SAC training (Sections 2.5–2.6) – _____
4. Results and robust policy (Sections 2.7–2.9) – _____
5. Discussion (Section 2.10) – _____
6. Python code and figures – _____
7. Animation (Manim) – _____

References

[1] M. W. Spong, "The swing up control problem for the Acrobot," *IEEE Control Systems Magazine*, vol. 15, no. 1, pp. 49–55, 1995.

[2] R. Tedrake, *Underactuated Robotics*, MIT, 2024. Available: https://underactuated.csail.mit.edu

[3] S. Skogestad and I. Postlethwaite, *Multivariable Feedback Control*, 2nd ed., Wiley, 2005.

[4] H. K. Khalil, *Nonlinear Systems*, 3rd ed., Prentice Hall, 2002.

[5] E. Coumans and Y. Bai, "PyBullet, a Python module for physics simulation," 2016–2024. Available: https://pybullet.org

[6] D. C. Youla, J. J. Bongiorno and C. N. Lu, "Single-loop feedback-stabilization of linear multivariable dynamical plants," *Automatica*, vol. 10, pp. 159–173, 1974.

[7] B. D. O. Anderson and J. B. Moore, *Optimal Control: Linear Quadratic Methods*, Prentice Hall, 1990.

[8] T. Haarnoja, A. Zhou, P. Abbeel and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. ICML*, 2018.

[9] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus and N. Dormann, "Stable-Baselines3: Reliable reinforcement learning implementations," *JMLR*, vol. 22, no. 268, pp. 1–8, 2021.